

Abstract Data Types; Singly Linked Lists: Nodes & Traversal

CSCI 235 — Class #7

Tong Yi

Hunter College, CUNY

Tuesday, September 22, 2026

Where We Are

- You have built `MiniVector`: a class that **owns** a heap array, grows it, copies it correctly (Rule of Three), and cleans up after itself.
- You have tested it and hunted its memory bugs with Valgrind.
- Exercise 3 (Big Three) is **due today**. Project 1B is due Th 10/1.

Today: a change of level

For six classes we asked *“how do I implement this correctly?”* Today we step back and ask *“what is this thing, independently of how I implement it?”* — and then build a completely different implementation of the same idea.

Roadmap for Today

- ➊ **Abstract Data Types:** interface vs. implementation.
- ➋ **Where the array wins:** two CSCI 135 games you have already written.
- ➌ **Why another list?** Where `MiniVector` is expensive, and why.
- ➍ **Nodes:** the building block — a value plus a pointer.
- ➎ **Traversal:** the loop you will write a hundred times this semester.
- ➏ **Sequential access** and what it costs.
- ➐ **Preview:** how this shows up in Project 2's `MazeSolver`.

What Is an Abstract Data Type?

An **abstract data type (ADT)** is a description of a collection in terms of **what you can do with it**, said without reference to *how* it is stored.

An ADT specifies

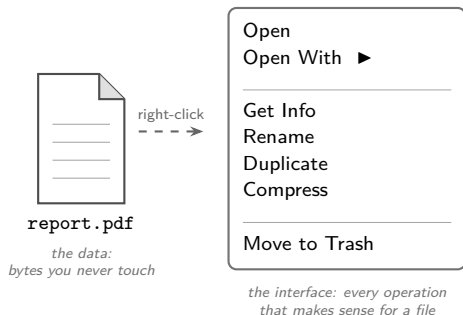
- the **operations** available (add, remove, look up, how many);
- what each operation **means** — its effect on the collection;
- what is **not allowed** (e.g. removing from an empty list).

An ADT does *not* specify

Arrays, pointers, capacity, new/delete, or any line of code. Those are the **implementation** — and one ADT can have many different implementations.

An Everyday Analogy: Right-Click a File

You never see the bytes on disk. You see a **menu of things you are allowed to do** — and the menu depends on *what kind of thing* you clicked.



Right-click a **folder** and the menu changes.

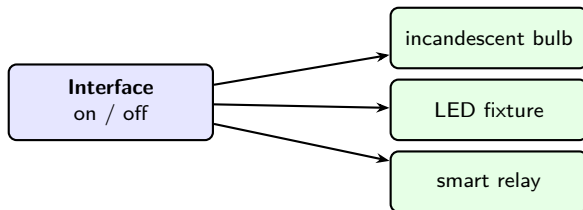
Same idea: *the data decides which operations are offered.*

This is exactly a C++ class

The **private data members** are the bytes — nobody outside the class may touch them. The **public member functions** are the menu. An **ADT** is that menu written down *before* anyone decides what the bytes look like.

An Everyday Analogy: The Light Switch

The **interface** of a light is two operations: *turn on*, *turn off*. Everyone in the building knows how to ask for them without knowing anything else — note that even “flip the switch up” is already one particular implementation.



The point

The bulb can be swapped for an LED without anybody having to learn anything new. The *interface* stayed fixed; the *implementation* changed. Cost changed too — the LED is cheaper to run — and that is exactly the trade-off we will make today.

The List ADT

A **list** is an ordered collection of items, accessed by position. A minimal interface, in the vocabulary the standard library already uses:

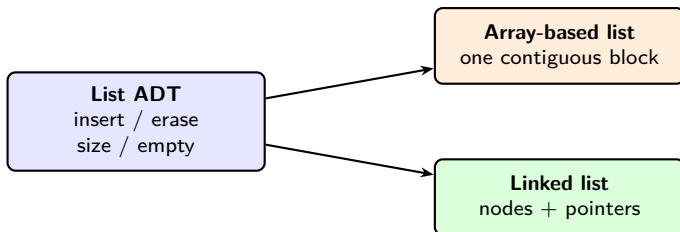
Operation	Meaning
<code>empty()</code>	is the list empty?
<code>size()</code>	how many items are in it
<code>insert(pos, item)</code>	put <code>item</code> at position <code>pos</code> ; later items shift back
<code>erase(pos)</code>	take out the item at <code>pos</code> ; later items shift forward
<code>getEntry(pos)</code>	the item at <code>pos</code> — not offered by <code>std::list</code>
<code>clear()</code>	remove everything

Two details we are glossing over: in the real `std::list`, `insert` and `erase` name the position with an iterator rather than an `int`; and “shift” above describes the positions being renumbered — in a linked list nothing in memory moves at all.

Two things to notice

No `capacity()`, no `reserve()`, no `data_` — those are `MiniVector`'s private business, not the List ADT's. And `std::list` simply **does not have** `getEntry`: there is no way to jump to a position, so the only route to item `pos` is to walk there. Our own `LinkedList` *will* offer it — at that price.

One ADT, Two Implementations

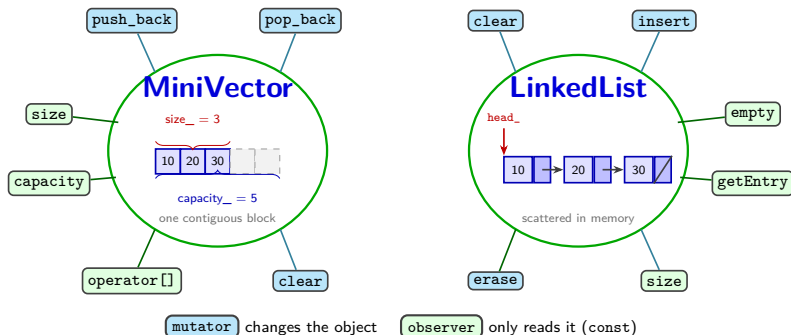


Both honor the same contract, so code written against the interface keeps working when you swap one for the other — but its **running time** changes, sometimes dramatically.

A word about MiniVector

MiniVector is where you learned how the left-hand box works, but its own interface is vector-like (`push_back`, `operator[]`) rather than the List ADT on the previous slide. “Same ADT” means *same behavior promised*, not *same C++ function names*.

Two Implementations, Drawn the Same Way



Read the two menus against each other

The middles are unrecognizable, yet most of the menu is the same — that is the ADT doing its job. Now look at the differences: `capacity` is a fact about an *array*, so no linked list offers it; and where the names *do* match, the price does not. `operator[]` is one addition. `getEntry(pos)` is a walk.

The Array Has a Cheap End: the Back

Before we look for a different implementation, it is worth being clear about what the array-backed one is *good* at.

Operation	Cost	Why
<code>push_back(v)</code>	amortized $O(1)$	writes one slot past the end; when full, reallocates and doubles , so reallocations get geometrically rarer
<code>pop_back()</code>	$O(1)$	decrement <code>size_</code> ; nothing moves
<code>back()</code>	$O(1)$	<code>data_[size_ - 1]</code>

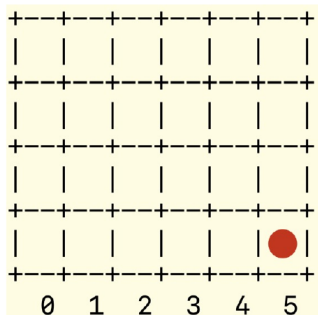
The observation that matters

All three touch only the **last** element, and a program that never inserts anywhere else pays none of the shifting cost we are about to discuss. You wrote two such programs in CSCI 135.

“Doubles” is not a detail (one line, from Day 4)

Growing by a *constant* instead — one slot, or even ten — makes n `push_backs` cost $O(n^2)$ in total. Only **geometric** growth buys amortized $O(1)$.

Connect 4 (CSCI 135, Spring 2025)



The board is six **bins**, each holding up to four pieces:

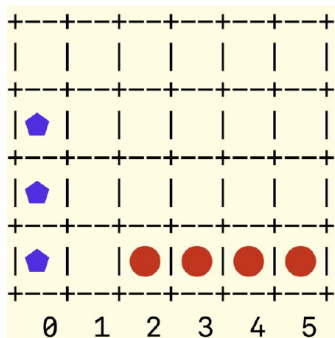
```
std::vector<std::vector<int>>> grid;
```

Each bin is one `vector<int>`; the `int` is the player id. Player red has just dropped a piece into bin 5.

Why the back, and only the back

Gravity does the work. A piece dropped into a column lands on top of whatever is already there — the *end* of that bin's vector. There is no such move as “insert a piece underneath the others.”

Connect 4: Four push_backs Later



The move that completed the red row was one line:

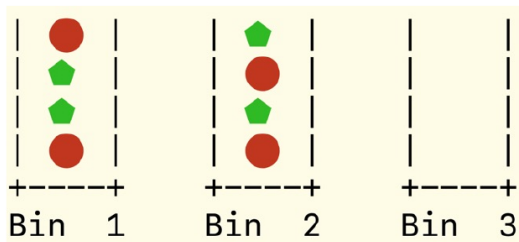
```
game->grid[2].push_back(0); // 0 illustrated by  
red circle
```

- `grid[2]` — the bin the player chose;
- `push_back(0)` — drop player 0's piece on top;
- `grid[2].size()` — how full that column is, which is also the piece's row.

No index arithmetic at all

Nobody had to compute “which row does this piece land in?” The vector's own `size()` answers that, because pieces can only ever accumulate at the back.

Shape Sort (CSCI 135, Fall 2024)



The same shape, a different game: sort the coloured shapes so that each bin holds only one kind.

```
std::vector<std::vector<int>> bins; // each bin: a vector<int> of shape ids
```

The rule of the game is a rule about vectors

“Only the top element of a bin can be moved.” The top of the drawn bin is the **back** of the vector — index 0 is drawn at the bottom. The game’s rule and `vector`’s cheap end are the same restriction.

Shape Sort: A Move Is pop_back then push_back

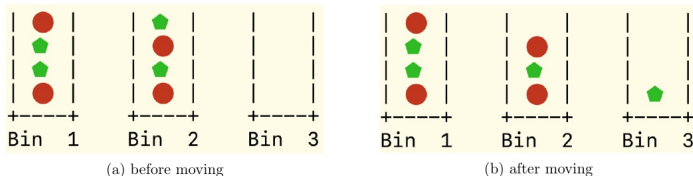


Figure 1: before and after moving from Bin 2 to Bin 3

Moving the green pentagon from bin 2 to bin 3:

```
int shape = bins[1].back(); // look at the top -- 0(1), no removal
bins[1].pop_back();         // take it off bin 2
bins[2].push_back(shape);   // drop it onto bin 3
```

Three operations, three $O(1)$ calls

Every legal move in the entire game is some number of repetitions of those three lines. Nothing is ever inserted into the middle of a bin, and nothing is ever removed from the bottom.

What Those Two Games Have in Common

Both store a growable collection, and both touch it in exactly one place:

	Connect 4	Shape Sort
Add	drop a piece into a column	move a shape onto a bin
Remove	(never)	take the top shape off a bin
Inspect	the column's height	the top shape
Vector calls	push_back	back, pop_back, push_back

Last in, first out

Add at the back, look at the back, remove from the back — that access pattern has a name, and a whole ADT of its own. It is the **stack**, and we build it on Tuesday 9/29.

So Why Would We Want Anything Else?

If the back is all you ever touch, an array-backed vector is the right answer and this lecture could stop here.

But look at what those two games never do

Neither one ever inserts a piece at the *front* of a bin, or removes one from underneath the others. That is not a coincidence in the game design — it is the operation the array is bad at.

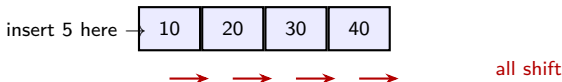
The rest of today

Plenty of problems *do* need to add and remove at the front, or in the middle. For those, the array's one cheap end is not enough — so we need a second implementation of the List ADT.

Where MiniVector Is Expensive

Two operations you have already written, at the front of the array:

- **Insert at the front:** every one of the n existing elements must shift one slot to the right first. $O(n)$.
- **Remove from the front:** every element after it shifts left. $O(n)$.

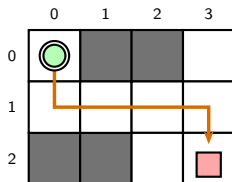


And one more cost

When the array is full, `reserve()` allocates a *new* array and copies everything — and every pointer or reference into the old array dangles (Day 6).

Who Actually Needs Insert-at-the-Front?

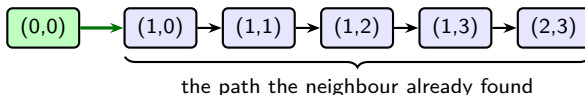
Front insertion is not contrived — it arises naturally when we build a path as a **recursive DFS unwinds**. Finding a way out of a maze: start at , reach the exit at .



The search works **backwards**: it wanders until it reaches the exit, and only on the way back out does each cell learn that it leads there.

So the exit is known *first*, the start *last*, and each cell on the way back joins the **front** of the path its neighbour already found.

added last



Why this is the array's worst operation

Each cell on the way back costs one insertion *at the front*: $O(1)$ on a linked structure, but on an array it shifts every cell found so far — $O(n^2)$ for the path. We write this code in Project 2.

The Idea Behind a Linked List

An array stores elements **next to each other** in memory, so “the next element” is an address one step away — fast to find, but expensive to make room in the middle.

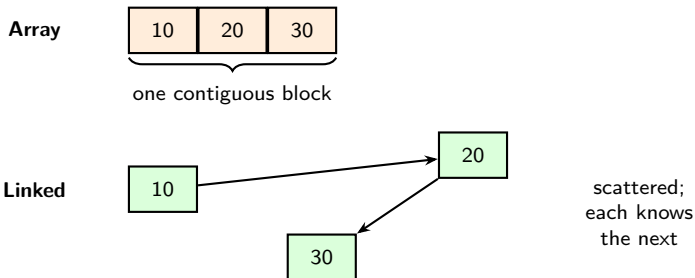
The alternative

Let the elements live **anywhere** in memory, and have each one *explicitly record* where the next one is. Once you have found the insertion point, making room costs nothing but rewriting one or two of those records — no element shifts, and nothing is reallocated. **Finding the point is a separate cost, and it is still $O(n)$** ; we return to that precisely at the end of class.

An analogy: a trail of clues

A shelf of books is an array: to reach the fifth book you walk straight to the fifth position. A treasure hunt is a linked list — each **clue is a node**, holding one thing you came for *plus where to find the next clue*. Adding a stop to the trail means rewriting one clue, not renumbering the others. But there is no way to reach the fifth clue without following the first four.

Memory Layout: Contiguous vs. Scattered

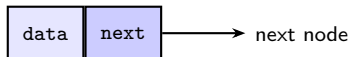


The array's positions are implied by arithmetic. The linked list's order is **stored**, in the pointers themselves — which is why it costs extra memory, one pointer per element.

The Building Block: A Node

A **node** bundles one piece of data with the address of the next node:

```
struct Node {  
    int data;           // the value this node stores  
    Node* next;         // address of the NEXT node, or nullptr at the end  
};
```



A struct that points to itself

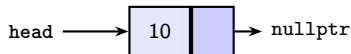
`Node* next;` is legal *inside* `Node` because the compiler knows the size of a **pointer** to `Node` without yet knowing the size of a `Node` object. A member `Node next;` would be recursive containment with no finite size, and does not compile.

A question to take with you

What if we want a linked list of `std::string` instead of `int` — or of `Coord`, the way the maze path does? Writing a fresh struct for every element type is not the answer, and you have already solved this problem once, in Project 1B.

Building a Three-Node Chain by Hand

```
Node* head = new Node{10, nullptr};  
head->next = new Node{20, nullptr};  
head->next->next = new Node{30, nullptr};
```



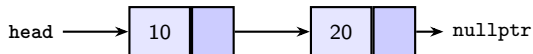
After statement 1: a one-node list. `head` holds the node's address, and the node's `next` is `nullptr`.

Two things worth naming

`head` is not a node — it is a single pointer variable holding the address of the first node. And the last node's `next` is `nullptr`: that, not a count, is how the list announces its own end.

Building a Three-Node Chain by Hand

```
Node* head = new Node{10, nullptr};  
head->next = new Node{20, nullptr};  
head->next->next = new Node{30, nullptr};
```



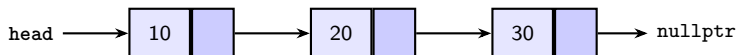
Statement 2 **overwrites** that `nullptr` with the address of the new node — the end of the list moves right.

Two things worth naming

`head` is not a node — it is a single pointer variable holding the address of the first node. And the last node's `next` is `nullptr`: that, not a count, is how the list announces its own end.

Building a Three-Node Chain by Hand

```
Node* head = new Node{10, nullptr};  
head->next = new Node{20, nullptr};  
head->next->next = new Node{30, nullptr};
```



Statement 3 reaches the second node's next (`head->next->next`) and overwrites that one in turn.

Two things worth naming

`head` is not a node — it is a single pointer variable holding the address of the first node. And the last node's `next` is `nullptr`: that, not a count, is how the list announces its own end.

head == nullptr Means Empty



An empty linked list is not an allocation of size zero and not a special flag — it is simply head holding `nullptr`.

Where the bugs will be

Every function you write from here on must answer one question first: *what happens when the list is empty?* Dereferencing `head->data` when `head` is `nullptr` is **undefined behavior**. It may crash, but C++ does not guarantee a clean or immediate failure.

. vs. ->: A Reminder

You met both on Day 3; here they sit side by side:

```
Node n;           // an object
Node* p = &n;      // a pointer to an object

n.data = 10;       // '.' on an OBJECT
p->data = 10;       // '->' on a POINTER == (*p).data
```

The single most common linked-list typo

`curr.next` where `curr` is a `Node*`. This one is friendly — it will not compile. The dangerous mistakes are the ones that compile happily and crash at run time.

. vs. ->: A Reminder

You met both on Day 3; here they sit side by side:

```
Node n;           // an object
Node* p = &n;     // a pointer to an object

n.data = 10;      // '.' on an OBJECT
p->data = 10;     // '->' on a POINTER == (*p).data
```

The single most common linked-list typo

`curr.next` where `curr` is a `Node*`. This one is friendly — it will not compile. The dangerous mistakes are the ones that compile happily and crash at run time.

~~`curr.next`~~

~~`curr->next`~~

The Traversal Loop — You Fill In the Blanks

To visit every node in order, we need an ordinary for loop. It has four slots. What goes in each?

```
void display(Node* head) {  
    for ( (1) ; (2) ; (3) ) {  
        (4)  
    }  
    std::cout << "\n";  
}
```

The Traversal Loop — You Fill In the Blanks

To visit every node in order, we need an ordinary for loop. It has four slots. What goes in each?

```
void display(Node* head) {  
    for ( Node* curr = head ; (2) ; (3) ) {  
        (4)  
    }  
    std::cout << "\n";  
}
```

- **(1) Where do we start?** At the first node — `Node* curr = head`. Note that we traverse with `curr`, never with `head` itself.

The Traversal Loop — You Fill In the Blanks

To visit every node in order, we need an ordinary for loop. It has four slots. What goes in each?

```
void display(Node* head) {  
    for ( Node* curr = head ; curr != nullptr ; (3) ) {  
        (4)  
    }  
    std::cout << "\n";  
}
```

- **(1) Where do we start?** At the first node — `Node* curr = head`. Note that we traverse with `curr`, never with `head` itself.
- **(2) When do we stop?** When there is no node left to look at — `curr != nullptr`.

The Traversal Loop — You Fill In the Blanks

To visit every node in order, we need an ordinary for loop. It has four slots. What goes in each?

```
void display(Node* head) {  
    for ( Node* curr = head ; curr != nullptr ; curr = curr->next ) {  
        (4)  
    }  
    std::cout << "\n";  
}
```

- (1) **Where do we start?** At the first node — `Node* curr = head`. Note that we traverse with `curr`, never with `head` itself.
- (2) **When do we stop?** When there is no node left to look at — `curr != nullptr`.
- (3) **How do we move on?** By following the link the node stores — `curr = curr->next`.

The Traversal Loop — You Fill In the Blanks

To visit every node in order, we need an ordinary for loop. It has four slots. What goes in each?

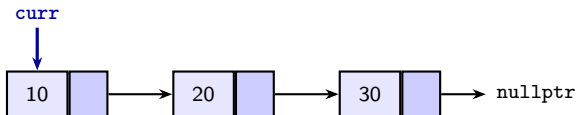
```
void display(Node* head) {  
    for (Node* curr = head; curr != nullptr; curr = curr->next) {  
        std::cout << curr->data << " ";  
    }  
    std::cout << "\n";  
}
```

- (1) **Where do we start?** At the first node — `Node* curr = head`. Note that we traverse with `curr`, never with `head` itself.
- (2) **When do we stop?** When there is no node left to look at — `curr != nullptr`.
- (3) **How do we move on?** By following the link the node stores — `curr = curr->next`.
- (4) **What do we do at each node?** The actual work — here, print `curr->data`.

This loop, or the while version of it, is the skeleton of nearly every linked-list function you will write.

Tracing the Loop

```
for (Node* curr = head; curr != nullptr; curr = curr->next) {  
    std::cout << curr->data << " ";  
}
```



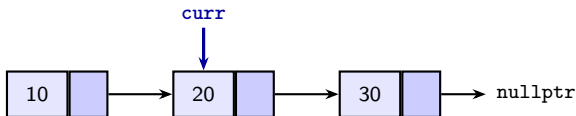
Console:

10

Iteration	curr	prints	curr = curr->next
1	node(10)	10	node(20)

Tracing the Loop

```
for (Node* curr = head; curr != nullptr; curr = curr->next) {  
    std::cout << curr->data << " ";  
}
```



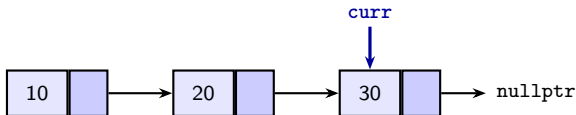
Console:

10 20

Iteration	curr	prints	curr = curr->next
1	node(10)	10	node(20)
2	node(20)	20	node(30)

Tracing the Loop

```
for (Node* curr = head; curr != nullptr; curr = curr->next) {  
    std::cout << curr->data << " ";  
}
```

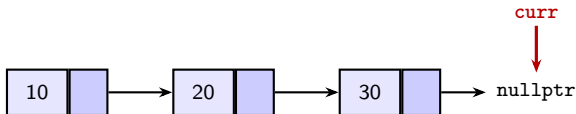


Console: 10 20 30

Iteration	curr	prints	curr = curr->next
1	node(10)	10	node(20)
2	node(20)	20	node(30)
3	node(30)	30	nullptr

Tracing the Loop

```
for (Node* curr = head; curr != nullptr; curr = curr->next) {  
    std::cout << curr->data << " ";  
}
```



Console:

10 20 30

Iteration	curr	prints	curr = curr->next
1	node(10)	10	node(20)
2	node(20)	20	node(30)
3	node(30)	30	nullptr
—	nullptr	loop ends	

The loop ends by *reaching* `nullptr` — the condition is tested, found false, and the body never runs again. `curr` is never dereferenced while it is `nullptr`.

Traversal Trap #1: Why We Traverse with curr

```
void display(Node* head) {           // head is a PARAMETER here
    while (head != nullptr) {
        std::cout << head->data << " ";
        head = head->next;           // correct HERE -- see below
    }
}
```

This version is correct

head here is a *parameter* — a local copy of the caller's pointer, so moving it does not disturb the caller's list.

Change one thing and it is a disaster

Write the same loop inside a **member function**, where head_ is a *data member*, and it walks the object's own head pointer to nullptr: every node is still allocated, but nothing points to the first — the whole list, leaked in four lines.

The lesson, stated generally

head = head->next is not wrong in itself. **Changing the pointer that *owns* the list, when you only meant to look at it, is wrong.**

Traversal Trap #2: Advancing Too Far

```
Node* curr = head;
while (curr->next != nullptr) { // stops one node EARLY
    std::cout << curr->data << " ";
    curr = curr->next;
}
```

Two different loops, two different jobs

- `curr != nullptr` — visit **every** node. This is what you want for printing, counting, searching.
- `curr->next != nullptr` — stop **on the last node**, with `curr` still pointing at it. This is what you want when appending. It crashes on an empty list unless you check `head` first.

Choosing the wrong one of these two is the classic off-by-one of linked lists — the exact analogue of `i <= size` on an array.

Counting the Nodes — Your Turn

Same loop, a different job: return how many nodes the list holds. Five slots to fill.

```
int size(Node* head) {  
    (1)  
    for ( (2) ; (3) ; (4) ) {  
        (5)  
    }  
    return count;  
}
```

Counting the Nodes — Your Turn

Same loop, a different job: return how many nodes the list holds. Five slots to fill.

```
int size(Node* head) {  
    int count = 0;  
    for ( (2) ; (3) ; (4) ) {  
        (5)  
    }  
    return count;  
}
```

- (1) A counter, starting at zero — `int count = 0;`

Counting the Nodes — Your Turn

Same loop, a different job: return how many nodes the list holds. Five slots to fill.

```
int size(Node* head) {  
    int count = 0;  
    for ( Node* curr = head ; (3) ; (4) ) {  
        (5)  
    }  
    return count;  
}
```

- (1) A counter, starting at zero — `int count = 0;`
- (2) Start at the first node — `Node* curr = head`

Counting the Nodes — Your Turn

Same loop, a different job: return how many nodes the list holds. Five slots to fill.

```
int size(Node* head) {  
    int count = 0;  
    for ( Node* curr = head ; curr != nullptr ; (4) ) {  
        (5)  
    }  
    return count;  
}
```

- (1) A counter, starting at zero — `int count = 0;`
- (2) Start at the first node — `Node* curr = head`
- (3) Stop when there is no node left to look at — `curr != nullptr`

Counting the Nodes — Your Turn

Same loop, a different job: return how many nodes the list holds. Five slots to fill.

```
int size(Node* head) {  
    int count = 0;  
    for ( Node* curr = head ; curr != nullptr ; curr = curr->next ) {  
        (5)  
    }  
    return count;  
}
```

- (1) A counter, starting at zero — `int count = 0;`
- (2) Start at the first node — `Node* curr = head`
- (3) Stop when there is no node left to look at — `curr != nullptr`
- (4) Advance along the link the node stores — `curr = curr->next`

Counting the Nodes — Your Turn

Same loop, a different job: return how many nodes the list holds. Five slots to fill.

```
int size(Node* head) {  
    int count = 0;  
    for (Node* curr = head; curr != nullptr; curr = curr->next) {  
        ++count;  
    }  
    return count;  
}
```

- (1) A counter, starting at zero — `int count = 0;`
- (2) Start at the first node — `Node* curr = head`
- (3) Stop when there is no node left to look at — `curr != nullptr`
- (4) Advance along the link the node stores — `curr = curr->next`
- (5) Count this node — `++count;`

$O(n)$ — and `MiniVector::size()` is $O(1)$

`MiniVector` keeps `size_` as a data member and just reports it; a bare chain of nodes has no such member, so the only way to know the length is to walk the whole list. A real `LinkedList` class stores an `itemCount_` alongside `head_` and updates it in every insert and remove — `size()` becomes $O(1)$, at the cost of one more invariant you must never let drift.

Searching the List — Your Turn

Same loop again. This time: return the node that holds target, or nullptr if no node does.

```
// Return the node holding target, or nullptr if it is not in the
// list.
Node* find(Node* head, int target) {
    for ( (1) ; (2) ; (3) ) {
        if ( (4) ) {
            (5)
        }
    }
    return nullptr;
}
```

Searching the List — Your Turn

Same loop again. This time: return the node that holds target, or nullptr if no node does.

```
// Return the node holding target, or nullptr if it is not in the
// list.
Node* find(Node* head, int target) {
    for ( Node* curr = head ; (2) ; (3) ) {
        if ( (4) ) {
            (5)
        }
    }
    return nullptr;
}
```

- (1) Start at the first node — Node* curr = head

Searching the List — Your Turn

Same loop again. This time: return the node that holds target, or nullptr if no node does.

```
// Return the node holding target, or nullptr if it is not in the
// list.
Node* find(Node* head, int target) {
    for ( Node* curr = head ; curr != nullptr ; (3) ) {
        if ( (4) ) {
            (5)
        }
    }
    return nullptr;
}
```

- (1) Start at the first node — `Node* curr = head`
- (2) Stop when there is no node left — `curr != nullptr`

Searching the List — Your Turn

Same loop again. This time: return the node that holds target, or nullptr if no node does.

```
// Return the node holding target, or nullptr if it is not in the
// list.
Node* find(Node* head, int target) {
    for ( Node* curr = head ; curr != nullptr ; curr = curr->next ) {
        if ( (4) ) {
            (5)
        }
    }
    return nullptr;
}
```

- (1) Start at the first node — `Node* curr = head`
- (2) Stop when there is no node left — `curr != nullptr`
- (3) Advance along the stored link — `curr = curr->next`

Searching the List — Your Turn

Same loop again. This time: return the node that holds target, or nullptr if no node does.

```
// Return the node holding target, or nullptr if it is not in the
// list.
Node* find(Node* head, int target) {
    for ( Node* curr = head ; curr != nullptr ; curr = curr->next ) {
        if ( curr->data == target ) {
            (5)
        }
    }
    return nullptr;
}
```

- (1) Start at the first node — `Node* curr = head`
- (2) Stop when there is no node left — `curr != nullptr`
- (3) Advance along the stored link — `curr = curr->next`
- (4) Does *this* node hold it? — `curr->data == target`

Searching the List — Your Turn

Same loop again. This time: return the node that holds target, or nullptr if no node does.

```
// Return the node holding target, or nullptr if it is not in the
// list.
Node* find(Node* head, int target) {
    for ( Node* curr = head ; curr != nullptr ; curr = curr->next ) {
        if ( curr->data == target ) {
            return curr;
        }
    }
    return nullptr;
}
```

- (1) Start at the first node — `Node* curr = head`
- (2) Stop when there is no node left — `curr != nullptr`
- (3) Advance along the stored link — `curr = curr->next`
- (4) Does *this* node hold it? — `curr->data == target`
- (5) Found it — stop early: `return curr;`

Complexity

Best case $O(1)$ (first node), worst case $O(n)$ (last node, or absent). **Sorting does not help:** binary search needs to jump to the middle, and a linked list cannot jump.

Searching: A Tempting Wrong Version

Here is a version students write every semester. It compiles, and on some inputs it even gives the right answer:

```
bool contains(Node* head, int target) {  
    for (Node* curr = head; curr != nullptr; curr = curr->next) {  
        if (curr->data == target) {  
            return true;  
        } else {  
            return false;    // <-- looks reasonable. It is not.  
        }  
    }  
    return false;  
}
```

What actually happens

The loop runs exactly once. Whichever way the `if` goes, the body returns — so `curr` is never advanced and no node after the first is ever examined. This function does not report whether `target` is *in the list*; it reports whether `target` is *the first element*.

The rule worth remembering

A search loop may return **early on success**, but it may only report failure *after the loop has finished*. Move the `return false;` out of the loop and the function is correct.

Destroying the List — Your Turn

Every node came from `new`, so someone must delete each one. Four slots — and here the *order* is the whole problem.

```
void clear(Node*& head) {  
    while ( (1) ) {  
        (2)  
        (3)  
        (4)  
    }  
}
```

Destroying the List — Your Turn

Every node came from `new`, so someone must delete each one. Four slots — and here the *order* is the whole problem.

```
void clear(Node*& head) {  
    while ( head != nullptr ) {  
        (2)  
        (3)  
        (4)  
    }  
}
```

- (1) Keep going while a node is left — `head != nullptr`

Destroying the List — Your Turn

Every node came from `new`, so someone must delete each one. Four slots — and here the *order* is the whole problem.

```
void clear(Node*& head) {  
    while ( head != nullptr ) {  
        Node* doomed = head;  
        (3)  
        (4)  
    }  
}
```

- (1) Keep going while a node is left — `head != nullptr`
- (2) Remember the node we are about to lose — `Node* doomed = head;`

Destroying the List — Your Turn

Every node came from `new`, so someone must delete each one. Four slots — and here the *order* is the whole problem.

```
void clear(Node*& head) {  
    while ( head != nullptr ) {  
        Node* doomed = head;  
        head = head->next;  
        (4)  
    }  
}
```

- (1) Keep going while a node is left — `head != nullptr`
- (2) Remember the node we are about to lose — `Node* doomed = head;`
- (3) Advance *first*, while we still can — `head = head->next;`

Destroying the List — Your Turn

Every node came from `new`, so someone must delete each one. Four slots — and here the *order* is the whole problem.

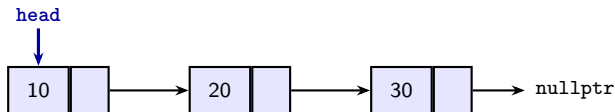
```
void clear(Node*& head) {  
    while ( head != nullptr ) {  
        Node* doomed = head;  
        head = head->next;  
        delete doomed;  
    }  
}
```

- (1) Keep going while a node is left — `head != nullptr`
- (2) Remember the node we are about to lose — `Node* doomed = head;`
- (3) Advance *first*, while we still can — `head = head->next;`
- (4) Now it is safe to free it — `delete doomed;`

Swap (3) and (4) and you have a use-after-free

`delete head;` followed by `head = head->next;` reads `->next` out of a node that was just freed — the dangling-pointer bug from Day 6, which Valgrind reports as an invalid read. This loop is also the body of your list's **destructor**.

Destroying the List, Step by Step

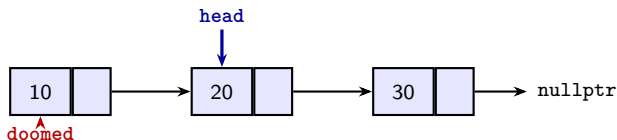


Before the loop: three nodes, head at the first.

Two pointers, and neither is optional

head says where the rest of the list is; doomed says which node still has to be freed. Drop doomed and you leak the node; free before advancing and you read freed memory.

Destroying the List, Step by Step

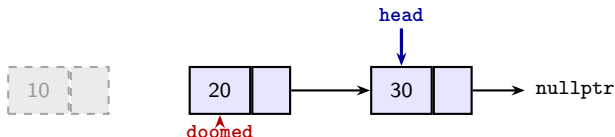


Iteration 1, after (2) and (3): `doomed` still holds node 10, and `head` has already moved on — so (4) is now safe.

Two pointers, and neither is optional

`head` says where the rest of the list is; `doomed` says which node still has to be freed. Drop `doomed` and you leak the node; free before advancing and you read freed memory.

Destroying the List, Step by Step



Iteration 2: node 10 is freed and gone. The same three steps again.

Two pointers, and neither is optional

head says where the rest of the list is; doomed says which node still has to be freed. Drop doomed and you leak the node; free before advancing and you read freed memory.

Destroying the List, Step by Step



Iteration 3: head is now nullptr, and node 30 is the last one to free.

Two pointers, and neither is optional

head says where the rest of the list is; doomed says which node still has to be freed. Drop doomed and you leak the node; free before advancing and you read freed memory.

Destroying the List, Step by Step



The condition is false, the list is empty — and every `new` has been matched by exactly one `delete`.

Two pointers, and neither is optional

`head` says where the rest of the list is; `doomed` says which node still has to be freed. Drop `doomed` and you leak the node; free before advancing and you read freed memory.

No Random Access

MiniVector::operator[] is one addition: data_[i] is the address data_ + i. A linked list has no such arithmetic — you must walk:

```
int getEntry(Node* head, int pos) {    // 0-based
    Node* curr = head;
    for (int i = 0; i < pos && curr != nullptr; ++i) {
        curr = curr->next;
    }
    if (curr == nullptr) { throw std::out_of_range("bad position"); }
    return curr->data;
}
```

The cost of a familiar-looking loop

```
for (int i = 0; i < n; ++i) { std::cout << getEntry(head, i); }
```

This looks like the array version and behaves like it — but each call walks from the front again, so the whole loop is $O(n^2)$. Traverse **once** with a curr pointer instead.

Array vs. Linked List: The Trade-Off

Operation	Array-based list	Singly linked list
Access by position	$O(1)$	$O(n)$
Insert / remove at front	$O(n)$ (shift all)	$O(1)$ (rewire head)
Insert at back	amortized $O(1)$	$O(n)$, or $O(1)$ with a tail pointer
Remove at back	$O(1)$	$O(n)$ — even <i>with</i> a tail pointer
Insert / remove in middle, <i>given the predecessor</i>	$O(n)$ (shift)	$O(1)$ (rewire)
Search for a value	$O(n)$	$O(n)$
Extra memory per element	none	one pointer per node
Grows by	realloc + copy all	one new Node

It is always the *predecessor* you need

Removing a node means rewriting the next of the node *before* it, and the links run only one way — so *finding* that predecessor is still $O(n)$. Same reason a tail pointer buys you insertion at the back but not removal: it reaches the last node instantly, never the one before it. Linked lists are fast at **rewiring**, not at **locating**.

So When Would You Pick Each?

Array-based, when ...

you index a lot; you mostly append and read; you care about cache behavior (contiguous memory is genuinely faster to scan than scattered nodes, by more than the Big-O suggests).

Linked, when ...

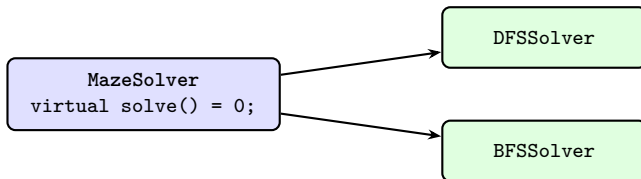
you insert and remove constantly at the front or at a position you already hold; you cannot afford an occasional expensive reallocation; you need the addresses of elements to stay valid as the collection changes.

That last point, stated precisely

Existing nodes **do not move** when other nodes are inserted or removed, so a pointer to a node stays valid *until that node itself is deleted*. That rules out the Day 6 bug, where `reserve()` relocated the array and invalidated a reference to an element nobody had touched. It does **not** mean linked lists are free of dangling pointers: delete a node while something still points at it and you have the same problem.

Where This Goes: Project 2's MazeSolver

Project 2 puts the two halves of this course so far together.



- **Day 2's polymorphism:** `MazeSolver` gives different solver implementations a common interface, so you call `solve()` without knowing which solver you hold — another example of programming to behavior rather than to representation.
- **Today's linked list:** the maze path is collected in a `std::list<Coord>`, and each step is added with `push_front()` as the recursion unwinds, so the path comes out in order without ever being reversed. After today you know why adding at the *front* is the natural, cheap operation for a linked structure.
- **Days 9–12:** the stack and the queue that make DFS and BFS differ — both of which you will build from the nodes you met today.

Your Questions From Thursday

Thank you for the slips — these came up on Thursday, 9/17, and the next several slides answer them.

- Two requests about **when slips are handed out**, and two about **when they come back graded**.
- **Valgrind**: what it actually does — and whether it is on the exam.
- More **edge-case examples**.
- A refresher on **templates** and on **copy constructors**.

Keep writing them

Several of these changed how today's class runs. That only works if you keep telling me what is unclear.

Your Request: Hand Out the Slips Earlier

Your requests

“Would it be possible to hand out lecture slips closer to the beginning of class? The shuffling towards end of lecture can be very distracting.” / “Hand out slips earlier, it is difficult to complete quickly.”

Done — starting today, the slips go out about **20 minutes into class**, instead of at the end.

Why 20 minutes and not the very start

By then we will have covered the first part of the day's material, so the questions are not a mystery when you get them — and you still have most of the class to fill the slip in as we go, rather than racing in the last five minutes. If a later question refers to something we have not reached yet, do not panic; it will be covered before the slips are collected.

Your Question: When Do Slips Come Back?

Your questions

“When will you upload our lecture slips?” / “My Lecture Slip 1 is ungraded!”

Slip grades and per-question feedback are being posted to **Gradescope**, and the backlog from the first weeks is being cleared now.

Two things to know

- A slip that shows as ungraded is **not** a zero — it means I have not reached it yet, and it still counts as attendance.
- With a class of this size, grading a set of slips takes some days. If a slip of yours is still missing after the next one is returned, email me — that usually means the scan did not pick up your name.

Your Question: What Exactly Is Valgrind?

Your question

“What exactly is the aspect and details of Valgrind?”

Valgrind runs your *already compiled* program inside an instrumented virtual machine. It does not look at your source code and it does not check your answers — it watches memory:

- every new / new[]: which line asked for the block, and how big it is;
- every delete / delete[]: whether that address was ever allocated, and whether it was already freed;
- every read and write: whether that byte is currently yours.

So the three reports mean exactly three things

“definitely lost” — allocated, never freed, nothing points at it (a leak). **“Invalid read/write”** — you touched memory that is freed or out of bounds (dangling pointer, off-by-one). **“Invalid free”** — you freed an address that was already freed (double free).

Your Question: Will Valgrind Be on the Exam?

Your question

“Will Valgrind be on the exam?”

No. Valgrind will not be on the midterm or the final — no flags to memorize, no output to reproduce.

Why we spent a class on it anyway

Because it is what you do before you hand something over. A leak produces correct output, so no test you write will catch it — and “it worked on my machine” is not the standard for shipped code. Running a memory checker over your own tests is the last step before you deliver a product and claim it is leak-free.

So it shows up in your work, not on the exam

Project 1 and Exercise 4 are graded partly on being memory-clean. The habit is the point: write the tests, run them under the checker, fix what it finds, *then* submit.

Your Question: More Edge-Case Examples

Your question

“When it came to the edge cases, could we go into more examples? I spend [time] trying to fix my own bugs on the programs.”

The useful habit is not memorizing a list of edge cases but knowing **where they come from** — every boundary is a place a condition flips:

```
// For ANY container, ask these four, in this order:  
// 1. What happens when it is EMPTY?      size_ == 0, head == nullptr  
// 2. What happens with exactly ONE element? first is also last  
// 3. What happens at the CAPACITY boundary? size_ == capacity_  
// 4. What happens when the thing is NOT FOUND / out of range?
```

Applied to today's material

Ask those four about a linked list and you get the empty list, the one-node list, and “value not present” — precisely the cases Thursday's operations break on. We will work through them explicitly next class.

Your Question: Templates Again, Please

Your question

“Could you go over the template <T> library again please?
Thank you!”

A template is a **recipe**, not code — the compiler writes the code once you name a type:

```
template <typename T>           // "T stands for some type"
T maxOf(const T& a, const T& b) { return (a > b) ? a : b; }
maxOf(3, 7);                    // compiler DEDUCES T = int, generates that version
maxOf(2.5, 1.5);               // compiler DEDUCES T = double, generates another

template <typename T>
class MiniVector { T* data_; /* ... */ };
MiniVector<int> v1;              // CLASS template: you must SAY the type
MiniVector<std::string> v2;     // a second, separate class is generated
```

Why the code has to live in the header

Nothing exists until a type is supplied, so the compiler must see the function bodies at the moment it hits `MiniVector<int>`. That is why a class template's implementation goes in the `.hpp` (or a `.tpp` included from it), never in a separate `.cpp` — otherwise you get undefined reference at link time.

Your Question: Writing Copy Constructors

Your question

“Understanding more on writing copy constructors.”

Every copy constructor for a class that owns heap memory does the same three things, in the same order:

```
MiniVector::MiniVector(const MiniVector& other)
: size_(other.size_), capacity_(other.capacity_) { // 1. the bookkeeping
  data_ = new int[capacity_];                      // 2. MY OWN array
  for (std::size_t i = 0; i < size_; ++i) {
    data_[i] = other.data_[i];                     // 3. the CONTENTS
  }
}                                                    // no delete[]: this object owned nothing before now
```

The one-sentence test

Afterwards, `this->data_` and `other.data_` must hold **different addresses**. Same address means you wrote a shallow copy, and two destructors will free one array. Exercise 3 (due today) is this pattern on a second class; Day 6's slides put it side by side with `operator=`.

FAQ (Again): Practice Exams

Your question

“Will we see the exam format to have an experience of it?
Like practice exams?”

No practice exam will be posted — same answer as Thursday, and worth repeating since it came up again:

- Course materials change every semester, so an old exam was written for a different sequence of lectures and projects.
- A posted practice exam sets the wrong expectation: students study its specific questions rather than the material.

What you *will* get

The format is described in the Midterm Review class (Th 10/15), and the **lecture slips are the best preview of the question style** — short, code-reading, “what is wrong here and why.” One of you wrote simply, “I really have to review on this.” That is the right instinct: review the slips.

What's Due

Item	Posted	Due
Exercise 3 (Big Three)	Tu 9/15	today, Tu 9/22
Project 1B: MiniVector	Tu 9/15	Th 10/1
Exercise 4 (Linked List Ops)	Th 9/24	Th 10/1

Before Thursday

Write `display()`, `size()` and `find()` yourself, from a blank file, and run them on three lists: empty, one node, three nodes. Thursday's material assumes the traversal loop is already automatic for you.

Key Takeaways

- An **ADT** says what a collection *does*; an implementation says how. One ADT, many implementations, different costs.
- A **node** is data plus the address of the next node; head points at the first node, nullptr marks the end.
- `head == nullptr` is the empty list — check it first, in every function.
- The **traversal loop** for `(Node* curr = head; curr != nullptr; curr = curr->next)` is the workhorse. Never walk `head_` itself.
- Access is **sequential**: position lookup is $O(n)$, and calling it inside a loop quietly costs $O(n^2)$.
- Linked lists are cheap to **rewire**, not to **locate** — and every node still has to be deleted.

Questions?

See you Thursday, 9/24 — Linked List Operations: Insert,
Delete & Reverse.

Backup Slides

Complete, uncondensed code for reference.

Backup: A Minimal LinkedList Class

```
class LinkedList {
public:
    LinkedList();                // head_ = nullptr, count_ = 0
    ~LinkedList();              // clear() -- deletes every node
    LinkedList(const LinkedList& other);    // deep copy
    LinkedList& operator=(const LinkedList& other); // Rule of Three

    bool empty() const;
    int size() const;           // O(1) using count_
    int getEntry(int pos) const; // O(n): must traverse
    void display() const;

private:
    struct Node {
        int data;
        Node* next;
    };
    Node* head_;
    int count_;
};
```

Backup: The Four Traversal Functions

```
void LinkedList::display() const {
    for (Node* curr = head_; curr != nullptr; curr = curr->next) {
        std::cout << curr->data << " ";
    }
    std::cout << "\n";
}

int LinkedList::size() const { return count_; }

bool LinkedList::empty() const { return head_ == nullptr; }

LinkedList::~~LinkedList() {
    while (head_ != nullptr) {
        Node* doomed = head_;
        head_ = head_->next;
        delete doomed;
    }
    count_ = 0;
}
```

Backup: Deep Copy (Rule of Three Preview)

```
LinkedList::LinkedList(const LinkedList& other)
: head_(nullptr), count_(other.count_) {
    if (other.head_ == nullptr) { return; } // copying an empty list

    head_ = new Node{other.head_>data, nullptr}; // first node
    Node* tail = head_;                          // end of OUR list
    for (Node* src = other.head_>next; src != nullptr; src = src->
        next) {
        tail->next = new Node{src->data, nullptr};
        tail = tail->next;
    }
}
```

Same principle as MiniVector

A shallow copy would leave two lists sharing one chain of nodes — and two destructors would delete each node twice. We will work through this properly when we build the full class.